Østfold University College

AI-assisted HS code Classification for E-Commerce Import

Khayam Nami (192732), Deniel Eggereide (70453), Shokoufeh Ghadersouri (211444), Vemund Vestgøte Brynjulfsen (213241), Kjell Randby Kristensen (202894)

February 22, 2026

Interdisciplinary Project Report
Østfold University College

Abstract

This study investigates the application of machine learning techniques for Optical Character Recognition (OCR) and Harmonized System (HS) code classification.

Today, the logistics company Jetcarrier manually assigns HS codes to orders.
This study evaluates the feasibility of automating HS code assignment through machine learning.

Large Language Models (LLM) are investigated for OCR to extract order descriptions from receipts. Several approaches for HS code classification based on text descriptions are also investigated. The investigated methods are Natural Language Processing (NLP), Large Language Models (LLMs), Semantic Similarity Comparison (SSC), and Classic Supervised Classification models. These approaches' strengths and weaknesses are showcased.

LLMs showed good results in extracting textual order descriptions from receipts.
These descriptions were then used with classification models to assign HS codes to the descriptions.

Experimental results indicate that NLP-based approaches outperformed other methodologies, being able to accurately assign up to 8-digit HS codes. Conversely, LLMs, SSC, and traditional supervised models faced challenges primarily due to data limitations in the dataset.

The study also emphasizes the critical role of high-quality, sufficient data for training effective classification models. Machine learning is known for needing a lot of data to accurately find patterns and generalize. A challenge was unsatisfactory data quality, a lack of enough HS code digit examples, and class imbalance.

NLP worked sufficiently across all digits of the HS code. Supervised machine learning worked well on the initial digits, but ceased to work on lower digits due to sparse data.

LLM and SSC struggled with HS code assignment, but can potentially work with less data than Classic Supervised Classification, and without pretraining, in addition to not needing labeled data, which may give less system maintenance.

It was found that machine learning is viable for automatically assigning HS codes, given that there is sufficient data.

Contents

Abstract **1**

1 Introduction **5**
1.1 Background and Context . . . 5
1.2 Problem statement . . . 5
1.3 Research Objectives . . . 6

2 Theoretical Background **7**
2.1 Optical Character Recognition (OCR) . . . 7
2.2 Classic Supervised Classification . . . 7
2.3 Large Language Models (LLMs) . . . 8
2.4 Natural Language Processing (NLP) . . . 9

3 Literature Review **11**
3.1 Search Procedure . . . 11
3.2 Literature Review Results - Automatic Information Extraction from Documents 12
3.3 Literature Review Results - HS Classification Methods . . . 13
3.3.1 Supervised Classification-Based Methods . . . 13
3.3.2 LLM-Based Methods . . . 14
3.3.3 NLP-Based Methods . . . 14
3.3.4 Semantic Similarity–Based Methods . . . 15
3.4 Conclusion Literature Review . . . 16

4 Experimental Setup **17**
4.1 Data Sources . . . 17
4.1.1 Norwegian Customs HS Code Database . . . 17
4.1.2 JetCarrier's Internal Tariff Data . . . 18
4.2 Part 1: Information Extraction Implementation . . . 18
4.2.1 Data Processing and Preparation . . . 18
4.2.2 OCR Pipeline and LLM-based Extraction . . . 19
4.2.3 Sampling Strategy and Benchmarking . . . 20
4.2.4 Structuring Data for Machine Learning Models . . . 21
4.2.5 Tools and Libraries . . . 21
4.2.6 OCR Pipeline Overview . . . 21
4.3 Part 2: HS Code Classification Implementation . . . 22
4.3.1 Classic Supervised Classification . . . 22
4.3.2 Semantic Similarity Comparison . . . 24
4.3.3 LLM Based System . . . 25
4.3.4 NLP Based System . . . 28

5 Results and Discussion **29**
5.1 Results . . . 29
5.1.1 Results for 3-digits . . . 29
5.1.2 Results for 6-digits . . . 30
5.1.3 Results for 8-digits . . . 31
5.1.4 Summary . . . 32
5.2 Discussion . . . 32

6 Conclusion and Future Work **34**
6.1 Conclusion . . . 34
6.2 Future Work . . . 34

References **34**

List of Tables

4.1 Software stack used in the OCR pipeline 21
4.2 Software stack used in the HS code matching pipeline 27

5.1 3-digit accuracy (%) by model and suggestion threshold 29
5.2 6-digit accuracy (%) by model and suggestion threshold 30
5.3 8-digit accuracy (%) by model and suggestion threshold 31

List of Figures

4.1 End-to-end pipeline for OCR and LLM-based information extraction from receipts 22
4.2 Supervised Classification Architecture 23
4.3 Product descriptions before (left) and after preprocessing (right). 25
4.4 An overview of the LLM system 25

5.1 Accuracy for 3-digit level for all models under Top 3, Top 5 and (for the LLM vector model) Top 50. 30
5.2 Accuracy for 6-digit level for all models under Top 3, Top 5 and (for the LLM vector model) Top 50. 31
5.3 Accuracy for 8-digit level for all models under Top 3, Top 5 and (for the LLM vector model) Top 50. 32

Listings

4.1 Prompt Generation for LLM-based Extraction 19
4.2 Key cleaning patterns. 26

Chapter 1

Introduction

1.1 Background and Context

JetCarrier AS is a Norway-based logistics company specializing in air and sea freight forwarding services from across the world to Scandinavia (JetCarrier, 2025). JetCarrier uses the international standard Harmonized System (HS) codes, consisting of six internationally defined digits and two Norway-specific digits, to classify incoming products and facilitate the customs clearance process according to import regulations, thereby streamlining logistics for its e-commerce clientele. With a terminal near Halden, JetCarrier has a strategic location that enables swift transport times throughout Scandinavia and the EU.

1.2 Problem statement

JetCarrier's logistics operations rely heavily on the manual classification of goods using HS codes to comply with customs regulations. This process is time-consuming and susceptible to errors resulting from incomplete or faulty information. Automating it would allow JetCarrier to increase efficiency, reduce delays, and reallocate resources to more complex classification tasks.

This project aims to investigate different methods for providing employees with probabilistic suggestions for HS code classifications. The system will utilize product descriptions from real-world freight forwarding orders as input data, and an HS code lookup table will be used to match and generate probabilistic predictions for classification. Using these suggestions, employees will have a starting point for the orders HS codes which they can refine through manual inspection.

By examining various methods for automated HS code classification, this project seeks to identify the most effective approach for generating initial predictions. These probabilistic suggestions will assist employees in streamlining the classification process while ensuring accuracy. The results will offer insights into the feasibility of AI-driven classification and its potential to optimize customs procedures.

1.3 Research Objectives

1. **Develop an OCR-based pipeline** to accurately extract product details from scanned receipts for use in HS code classification.
2. **Evaluate different NLP/LLM approaches** for filtering, structuring, and enhancing extracted product descriptions.
3. **Investigate the feasibility of real-time HS code classification** based on product descriptions.

Chapter 2

Theoretical Background

2.1 Optical Character Recognition (OCR)

Optical Character Recognition (OCR) is a technology used to convert scanned or image-based text (whether handwritten, typed, or printed) into an editable digital format. The OCR process consists of several stages: preprocessing, segmentation, feature extraction, classification, and post-processing. Preprocessing improves the quality of the scanned image to clarify characters and make segmentation more effective. This is achieved through various image processing techniques, such as noise removal, skew correction, thinning, normalization, and converting the image to grayscale or binary format. After preprocessing, segmentation divides the image into lines (rows of text), words (groups of characters), and characters (individual letters or symbols), preparing them for further analysis. Next, feature extraction focuses on identifying key patterns from the segmented characters that help distinguish one symbol from another. Based on these patterns, classification assigns each character to its correct category using structural, statistical, or syntactic methods, including models like neural networks. Finally, post-processing involves the last steps, such as grouping symbols, detecting errors, and applying corrections to improve the accuracy of the recognized text (Awel & Abidi, 2019; Islam et al., 2017).

Tesseract is a widely adopted open-source OCR engine. In Python, it can be accessed through the pytesseract library and is often used in combination with OpenCV. In this project, pytesseract was used to extract raw text from receipt images. To improve the performance of the OCR process, some preprocessing steps are required. These steps can be performed using OpenCV or similar Python libraries. After preprocessing, pytesseract recognizes and returns the extracted text as a string. Post-processing can then be applied to refine the results, using additional tools or libraries if needed.

2.2 Classic Supervised Classification

Supervised classification is a type of machine learning algorithm that learns to predict labels by training on data with predefined output classes. During training, it minimizes the difference between it's predictions and the given true labels. This makes the model able to generalize to make accurate predictions on unseen data.

These supervised classification methods were tried: Support Vector Machine (SVM), Random Forest (RF), and neural networks (NN) in the form of a Feed-Forward Neural Network (FFNN) and a Convolutional Neural Network (CNN)

SVM separates data points by creating a hyperplane between them. The model is trained to create hyperplane with the highest possible margin or distance away from the data points on either side of the hyperplane.

Random forest is an algorithm that uses multiple decision tree methods in an ensemble. A decision tree splits data into branches, leading to leaf nodes that represent a class. Random forest aggregates multiple decision tree predictions, where each tree's prediction is a vote for that prediction as the final output.

A neural network consists of many weights that are connected to each other. The weights are adjusted during training, where the model learns by minimizing the prediction error, by comparing its output with a true class.

One type of network is the feed-forward network. It consists of several layers of weights, where data flows in one direction, from the input layer, through hidden layers, and finally to an output layer.

A CNN adds convolutional layers to the network architecture. The convolutional layers are similar to grid structures, which makes it convenient for things like images and embeddings. The convolutional layers apply filters that slide over the input data and detect patterns.

For HS code classification, the problem can be approached as a hierarchical classification problem, as the HS codes lie in a hierarchy.

HS code classification can be thought of as a hierarchical classification (HC) problem. In a hierarchical classification problem, the labels can be thought of as lying in a hierarchical tree structure. It can be necessary to use several models across different parts of the hierarchical structure.

Costa et al. (2023) presents a comprehensive review of the literature on hierarchical text classification. It conducts several HC approaches and classification methods on various datasets.

There are several ways to structure a HC problem. One approach is to ignore the hierarchy by flattening the it. Then there is the global approach, where the hierarchy is taken into account, with classification done by a single model. There is the local approach, that uses multiple models, where each model only takes into account local information in the hierarchy.

A top-down approach is used during testing. It starts at the top of the hierarchy and uses several local classifiers to travel down the hierarchy, depending on which branch is classified as the correct class, until the end of the hierarchy is reached.

How the models are laid out across the hierarchy depends on the hierarchy structure. Local classifier per node (LCN) trains a binary classifier for each child node. Local classifier per parent node (LCPN) trains a multi-class classifier per parent node. Local classifier per level (LCL) trains a multi-class classifier for an entire hierarchy level.

2.3 Large Language Models (LLMs)

Large Language Models (LLMs) are advanced deep learning systems developed for understanding and generating human language. These models are widely used in applications such as question answering, summarization, information extraction, classification, and translation. They are based on the transformer architecture, which leverages self-attention mechanisms to effectively capture contextual relationships between words. This design enables LLMs to process long and complex texts more effectively than earlier models like recurrent neural networks (RNNs), which often struggled with long-range dependencies (Raiaan et al., 2024).

The training of LLMs typically follows a two-phase process. Initially, they undergo self-supervised pretraining on vast amounts of unlabeled text data to learn general language patterns. In the subsequent fine-tuning phase, the models are trained on smaller, labeled datasets tailored to specific tasks. This allows them to apply their pre-trained knowledge to meet user needs. This approach enables strong performance across tasks with minimal human annotation (Yiqiu Shen, 2023).

A particularly powerful feature of LLMs is their ability to perform in-context learning, where they generate appropriate responses based on input prompts containing task instructions and examples, without any additional parameter updates. This capability enables few-shot and zero-shot learning, where the model can generalize to new tasks with few or no examples. GPT-3 demonstrated impressive performance in such settings by leveraging in-context learning. Another important technique is instruction tuning, where the model is fine-tuned on datasets framed as natural language instructions. InstructGPT is a model trained in this way and further optimized using Reinforcement Learning from Human Feedback (RLHF). This technique refines the model's behavior by aligning it with human preferences through iterative feedback (Zhao et al., 2025).

LLMs were used in two main parts of this project: extracting information from OCR text and performing HS code classification. The models relied on in-context learning in both cases. They generated appropriate responses based on natural language prompts without updating their internal parameters.

2.4 Natural Language Processing (NLP)

Natural Language Processing (NLP) is a subfield within artificial intelligence that allows for the machine to, to a degree, interpret and generate text. At the core, modern NLP models are transformer-based, such as within BERT, which we are using in this study. With the usage of the self-attention mechanisms, it can capture contextual relationships between words in a sentence. This makes it ideal for situations when one wants to transform unstructured texts into structured representations. In the context of this study, where we are performing HS code classification, NLP is used to interpret the product descriptions and match them to the structure of Norwegian Customs. These descriptions often contain domain-specific terminology and spelling errors, and abbreviations introduce ambiguity to the texts. Transformer-based NLP models, which are trained on extremely vast amounts of text corpora and then later fine-tuned to domain-specific data, are well-tuned to this kind of task where ambiguity is a regular occurrence.

BERT, the NLP model of this study, is pre-trained through masked language modeling, where it learns to predict missing words based on both the left and right contexts. This kind of bidirectional training enables BERT to develop semantic representations of text. For HX classifications, tuning the model will allow it to understand product context from incomplete or vague inputs.

Furthermore, NLP models can be combined with semantic search, classification layers, or fine-tuned on labeled product HS code datasets. Studies such as those done by Anggoro et al. (2023) and de Lima et al. (2022) demonstrate that BERT shows promising results when performing HS code classification, especially when it's fine-tuned for local datasets.

In this project, we leverage BERT to classify products based on OCR-extracted descriptions from invoices. This involves text preprocessing, tokenization, and embedding, followed by training a classification head to predict the corresponding HS code. The application of NLP in this system enables it to generalize across diverse linguistic inputs and make informed predictions even when product descriptions are short, noisy, or ambiguous.

Chapter 3

Literature Review

This chapter reviews relevant research to support JetCarrier's goal of automating HS code classification. Two core topics are covered:

1. **Research Topic 1: Information Extraction** This topic investigates methods for converting unstructured documents (e.g. receipts, invoices) into structured data.
2. **Research Topic 2: HS Code Classification** This topic explores automatic assignment of Harmonized System (HS) codes based on product descriptions using machine leanring techniques.

We used a comprehensive set of search terms to capture relevant literature across both research areas.

1. **Information extraction**
   "Document Image Analysis", "Information Extraction", "Invoice Extraction", "Key Information Extraction", "Multilingual OCR", "Named Entity Recognition", OCR, "Optical Character Recognition", and "Receipt Processing".
2. **HS code classification**
   "Convolutional Neural Networks", CNN, "Customs Classification", "Decision Tree", "Deep Learning", "Embedding", "Embedding Comparison", "Harmonized System", "Hierarchical Classification", "Hierarchical Semantic Similarity", HS Code, "HS Code Automation", "HS Code Prediction", LLM, "Large Language Models", "Machine Learning", "Multi-class Classification", "Neural Networks", "Product Code Classification", "Product Description Matching", "Random Forest", "Semantic Similarity", "Sentence Embeddings", SVM, "Supervised Learning", "Support Vector Machine", "Tariff Classification", "Text Embedding", "Transformer Models", "Vector-based Classification", and XGBoost.

3.1 Search Procedure

We limited the search to peer-reviewed publications from 2019 to 2025 written in English or Norwegian. Results were screened by title, abstract, and keywords for relevance to the research topic. Relevant papers were then reviewed in full to ensure alignment with the research topics.

The searches were conducted on the following five research repositories, which were selected due to their coverage of peer-reviewed publications in fields relevant to the research topic. IEEE Xplore (Institute of Electrical and Electronics Engineers, 2024), Springer (Springer, 2024),

ScienceDirect (Elsevier, 2024), ACM Digital Library (Association for Computing Machinery, 2024), and Google Scholar (Google, 2024) were chosen for their coverage of research in artificial intelligence, document processing, and classification methods. The following academic sources were selected for their coverage of research in artificial intelligence, document processing, and classification methods:

- ACM Digital Library (2024)
- Google Scholar (2024)
- IEEE Xplore (2024)
- ScienceDirect (2024)
- Springer (2024)

3.2 Literature Review Results - Automatic Information Extraction from Documents

A crucial prerequisite for HS code classification is the accurate extraction of product information from unstructured documents such as receipts and invoices. Several studies have combined OCR technology with machine learning and language models to structure such information effectively.

Salvatore Rollino Subiabre (2024) introduced a system for automatic data extraction from Swedish receipts using image processing, OCR, and LLMs. The YOLOv8 model was used to detect the receipt area, and text was extracted using both Tesseract and EasyOCR. The most accurate output was passed to GPT (via OpenAI), which structured the data into fields such as item name, price, date, and tax. The study reported an OCR accuracy of approximately 82%, highlighting that LLM-based structuring performed well when the OCR output was of sufficient quality.

Loukil et al. (2024) extended this approach to both receipts and invoices by proposing a two-stage pipeline consisting of OCR and LLMs. Tesseract and DocTR were used for text extraction, while language models such as LLaMA 3 and Mistral 7B structured the data. Evaluation on the SROIE and FATURA datasets showed strong performance, with F1-scores up to 0.95 and accuracy up to 92%, all without the need for fine-tuning.

Ha & Horák (2022) focused on multilingual invoices in their system, OCRMiner, which combines OCR with rule-based text and layout analysis. Using BERT-based Named Entity Recognition and fuzzy matching, the system extracted six key fields despite layout variation, OCR errors, and multilingual complexity. The system achieved 90.5% accuracy on English invoices and 88.2% on Czech ones.

These studies demonstrate that combining OCR with language models and layout-aware processing provides a strong foundation for extracting structured product descriptions, which are essential for downstream classification into standard coding systems such as HS.

3.3 Literature Review Results - HS Classification Methods

Several studies have explored how machine learning and language models can automate HS code classification based on product descriptions. This section reviews four main approaches: supervised classification, large language models (LLMs), transformer-based NLP methods, and semantic similarity techniques. Each method uses different strategies to map text descriptions to the correct HS codes.

3.3.1 Supervised Classification-Based Methods

There are many studies that have explored using classic supervised classification models to automate HS code assignment. These are models such as Support Vector Machine (SVM), Tree-based algorithms like Decision Trees (DTs), XGBoost, and also neural networks (NN) for classification, such as convolutional neural networks (CNN). In these cases, classification models are trained assigning different layers of HS codes to product descriptions.

Ruder (2020) master's thesis explores classifying HS codes using supervised classification models. The thesis aims to classify six-digit HS codes required in the EU based on textual cargo descriptions. The data consists of one million US import cargo descriptions representing 3.243 unique HS codes. Feature vectors are generated using TF-IDF, Doc2Vec, Word2Vec, and GloVe. These vectors serve as input for various machine learning models, including Multinomial Logistic Regression, Multinomial Naive Bayes (NB), DTs, Random Forest, SVM, and CNNs. A deep neural network (DNN) model utilizing TF-IDF achieves the highest F1 scores of 61% and 62%, which outperforms manual human classification.

Altaheri (2019) explores a similar task. It applies six machine learning models to predict Harmonized System (HS) codes. The dataset is based on user-provided descriptions, with 22 million records from the Customs Department in Dubai, UAE. The models evaluated and compared include NB, K-Nearest Neighbors, DTs, Random Forest, Linear SVM, and AdaBoost. Among these models, SVM achieve the best performance in all evaluation metrics, including precision, recall, and the F1 score. It achieves up to 76.3% accuracy for predicting entire HS codes.

Paramartha et al. (2021) conducts a similar study to Altaheri (2019). It develops classification models for HS code prediction. The paper uses Indonesian import data, which consists of 1.3 million records. Six models are developed. These are Multinomial NB, K-Nearest Neighbor, DTs, Random Forest, Linear SVM, and Adaboost. The texts are represented using TF-IDF. The models are compared over several metrics. Linear SVM has the best performance for up to the first 4 digits of the HS code, while Decision Tree has the best performance when considering the whole 8 digit HS code.

These studies show that supervised classification models are highly applicable to the domain of HS code assignment.

3.3.2 LLM-Based Methods

Several recent studies have explored the use of large language models (LLMs) for automatic product classification in standardized systems such as HS, HTS, and UNSPSC. These approaches share the common goal of interpreting free-text descriptions and mapping them to appropriate codes without extensive training.

Oliveira et al. (2024) proposed SLIM-RAFT, a lightweight fine-tuning method applied to a small LLM (TeenyTinyLLaMA), for classifying products using Brazil's NCM codes, a national extension of the HS system. By combining question-answer training with explanation-based examples (chain-of-thought), the model achieved an average accuracy score of 8.63 out of 10, outperforming GPT-4 in this domain-specific task.

In a comparative study, Marra de Artiñano et al. (2023) examines the use of both traditionally used machine learning models and large language models (LLMs) for HS code classification based on product descriptions. Models such as SVM, Decision Tree, Random Forest, and Naive Bayes were evaluated alongside GPT-3.5, which was used with zero-shot prompting and no fine-tuning. The models were trained on customs data from Chile and tested on datasets from Paraguay and the U.S. Results showed that traditional models performed well on training data but failed to generalize, with accuracy dropping to around 30% at the 6-digit level. In contrast, GPT-3.5 achieved significantly better performance, reaching 60–80% accuracy at the 6-digit level, 70–80% at the 4-digit level, and 80–90% at the 2-digit level. The study highlights GPT-3.5's robustness and generalization ability across different languages and product domains.

Navasardyan (2024) investigated GPT-3 for classifying products into HTS codes (the U.S. extension of HS, consisting of 6 HS digits + 4 country-specific digits). The study used few-shot prompting, improving top-1 accuracy from 23% with generic prompts to 73% with well-designed examples. The model demonstrated strong generalization to unseen product descriptions and produced interpretable classification outputs.

Finally, Singh & Diao (2024) used GPT-4 to classify products into UNSPSC codes based on product descriptions from 50,000 public procurement entries in California. Through three prompt types (generic, cloze-style, and engineered), the model achieved strong results without additional training. Although the focus was on the UNSPSC system, the authors emphasized that the approach is transferable to HS classification tasks.

Collectively, these studies show that LLMs, particularly when guided by effective prompting, can achieve high precision, interpretability, and generalization in product classification, which makes them well suited for automated HS coding tasks.

3.3.3 NLP-Based Methods

Natural Language Processing (NLP) is a subfield within artificial intelligence that uses machine learning to allow an AI model to recognize and, to a certain extent, understand human language. BERT (Bidirectional Encoder Representations from Transformers) is a model that works in a "bidirectional" manner by trying to capture the meaning of a word based on the surrounding text.

In the study done byAnggoro et al. (2023), they explored the usage of transformer-based models

like the previously named BERT and distilled versions of this model, such as DistilBERT, to automatically assign HS codes from description texts on goods. In their attempt to work with these models, they found out that some of the challenges they faced stemmed from things such as short text lengths, ambiguity, technical jargon, proprietary feature namings, and lastly misspellings. In their results, they achieved an F1-score of 0.8561 from BERT and DistilBERT, achieving a score of 0.7643. While quite a difference in their scores, Anggoro and their team also note that DistilBERT has an advantage in being a smaller model, which would allow for deployment in a more resource-constrained environment.

de Lima et al. (2022) on the other hand compared language-specific BERT models, in this case a Portuguese one, against the multilingual version for classifying the goods and found through their experiments that the language-specific models exceeded multilingual versions by a notable amount. In their conclusion of the experiment, they highlighted the importance of tailored models for regional trade languages.

These studies illustrated the possibility of using transformer-based NLP models like BERT to automate HS code classifications, along with the challenges and the different applications of an NLP-based approach.

3.3.4 Semantic Similarity–Based Methods

Semantic Similarity-Based Methods compare the semantic closeness of descriptions to identify the most appropriate HS codes. The distances are determined by converting the semantic meaning of descriptions to dense vectors (often referred to as embeddings) and then calculating the distance between the vectors, e.g., the cosine of the vectors. Since this process does not depend on any historical dataset, these methods require no retraining and are robust to new and uncommon products.

Stassin et al. (2023) conducted a comparative study of two approaches to HS code prediction that rely on sentence embeddings generated by various pre-trained transformer-based models. In the first approach, embeddings were compared to those of previously validated customs declarations using distance metrics such as cosine, Euclidean, and Manhattan distances. HS codes were then recommended based on the most semantically similar matches, ranked by proximity. The second approach used the embeddings as input features in a supervised neural network, which was trained to directly classify the correct HS code. Both approaches were tested on the same dataset of 95,903 custom declarations with *top-k accuracy*, indicating whether the correct HS code appears amongst the top k most similar results. A semantic similarity configuration achieved the best top-3 accuracy of 89% and top-5 accuracy of 94.8% using the embedder MP-Net with cosine similarity. In comparison, the best-performing supervised model reached an accuracy of 35.3% top-3 accuracy.

Spichakova & Haav (2020) expanded upon the basic semantic similarity-based approach to address the challenge of short and often ambiguous descriptions in the HS structure. In this study, vector representations of the product descriptions were generated using the Doc2Vec model from Gensim, and distance was measured with cosine similarity. However, they incorporated a second component that captures the hierarchical taxonomy of the HS system using Wu & Palmer semantic similarity measure. This combined similarity score reflects not only how closely a product description matches another in text, but also how closely their corresponding HS codes are positioned within the HS taxonomy. Their results showed that 78.74% of HS codes were matched exactly, and another 5.48% were nearly correct, an improvement of 25.86% over the text-only approach.

These studies demonstrate that similarity-based methods offers robust, sclable, accurate and low-maintenance solutions to HS classification.

3.4 Conclusion Literature Review

We examined research on extracting structured data from unstructured documents and research classifying product descriptions using machine-learning approaches in this literature review.

There is a broad consensus across the literature that combining OCR with language models is effective for extracting product information from receipts and invoices. There are significant challenges related to input quality and document variability.

Both traditional supervised models and more recent transformer-based approaches have shown strong results for classification but require large datasets of high quality. LLMs, on the other hand, are valued for their generalization and flexibility, especially in low-data or cross-domain settings. NLP-based methods that use language-specific transformer models outperform multilingual variants and especially distilled models can deliver competitive accuracy with lower computational cost.
Semantic similarity-based methods are promising low-maintenance alternatives, outperforming traditional supervised methods, and do not require large datasets but do require data with higher semantic quality.

No single method was identified as universally optimal across all contexts. The literature highlights trade-offs between accuracy, generalizability, scalability, and interpretability among the different methods. We will build upon these findings in this project by testing and comparing multiple approaches on the real-world dataset provided by JetCarrier. We aim to contribute practical insights into how different methods perform under realistic conditions and help bridge the gap between academic research and operational AI systems for HS code classification.

Chapter 4

Experimental Setup

This chapter outlines the experimental implementation of the project and is divided into three main parts. First, the data sources are presented. Next, the information extraction pipeline is described, where OCR and language models are used to extract product descriptions from receipts. Finally, the setup for HS code classification is detailed, covering methods such as classical machine learning, semantic similarity, large language models (LLMs), and fine-tuned NLP models.

4.1 Data Sources

To develop models for real-world HS code classification, it is essential to have both large datasets of real-world product descriptions linked to their correct HS codes and a definitive reference that maps product descriptions to those HS codes. This section provides an overview of the data sources used in this study.

4.1.1 Norwegian Customs HS Code Database

The Norwegian Customs maintains a JSON lookup table of HS codes and descriptions of the products that belong to each HS code Tolletaten Norwegian Customs (2025). This lookup table will be used as a reference for validating and standardizing HS code predictions generated by the AI model.

The table provides a structured hierarchy of classifications, linking each HS code to product descriptions at different levels of specificity, from general sections to precise commodity codes. By integrating this resource, the model can cross-reference predicted classifications with official Norwegian tariff definitions, thereby reducing misclassification errors.

Furthermore, the structured format of the JSON file allows efficient querying and integration with Natural Language Processing (NLP) models, enabling direct comparisons between free-text product descriptions and official HS code definitions. This facilitates improved classification accuracy, especially in cases where product descriptions are ambiguous or inconsistent.

The hierarchical structure was flattened to enable faster lookups and more efficient integration with machine learning models, allowing direct mapping between product descriptions and their corresponding HS codes without requiring recursive traversal of multiple classification levels.

4.1.2 JetCarrier's Internal Tariff Data

JetCarrier maintains an internal database of tariff data including order numbers, customer IDs, receipts, customer-provided product descriptions, and HS codes for each order.

JetCarrier supplied **51,276** matched receipt–tariff files, consisting of **21,683** sea freight receipts and **29,593** air freight receipts. The receipts are a mix of screenshots of digital receipts, PDF files, smartphone pictures, scanned documents, and handwritten receipts. Due to these varying formats, differences in resolution and readability arise, necessitating the use of Optical Character Recognition (OCR) and additional filtering to accurately extract product descriptions.

4.2 Part 1: Information Extraction Implementation

This part outlines the main steps used to extract information from receipt data, including preprocessing, OCR, language model extraction, and model selection for full dataset processing.

4.2.1 Data Processing and Preparation

This section outlines the complete pipeline for data retrieval, cleaning, and structuring—highlighting both the technical workflow and the challenges encountered when handling a large, heterogeneous dataset.

4.2.1.1 Data Retrieval and Pre-processing

The dataset was provided by **JetCarrier** and consists of 51,206 receipts related to air and sea deliveries. Each receipt is available either as a PDF file or in various image formats (e.g., screenshots, phone photos, scans). Accompanying each receipt is a corresponding text (`.txt`) file containing metadata for JetCarrier's internal handling.

Initial Pre-processing

Due to the lack of a common formatting standard, the retrieval process necessitated a flexible approach:

- **File Conversion:** PDFs were converted to images using tools such as `pdf2image` to facilitate consistent OCR processing.
- **Metadata Extraction:** A custom routine parsed each receipt's corresponding metadata file, extracting key information such as tracking IDs and preliminary or final HS codes, as assigned by JetCarrier.
- **Handling Heterogeneity:** The wide range in image quality and formatting (ranging from low-resolution scans to varied screen captures) required pre-processing to standardize inputs for subsequent analysis.

4.2.1.2 Text and Image Data Cleaning

To ensure high-quality input and output in the OCR pipeline, two types of cleaning procedures were applied. First, image-specific preprocessing steps were conducted to optimize conditions for OCR. After the OCR process, the extracted text was cleaned and structured to prepare it for input to the LLM-based extraction stage.

- **Image Cleaning:** Given the variability in receipt images, several image-specific cleaning steps were applied. The images were rotated based on EXIF metadata and converted to

RGB to optimize conditions for OCR.

- **Text Cleaning:** Following OCR, the extracted text underwent additional processing to enhance structure and readability:
  - **Artifact Removal:** Characters such as form feed (`\f`) and carriage return (`\r`) were removed to ensure a cleaner, uniform text input.
  - **Line Structuring:** The cleaned text was segmented into lines, preparing it for the subsequent extraction of product information.

4.2.2 OCR Pipeline and LLM-based Extraction

Loukil et al. (2024) provide evidence that an LLM-centric extraction pipeline can structure and classify key information from invoices and receipts even with layout variations and OCR errors.

Hence, the extraction process was implemented using a combination of the following steps:

- **OCR Extraction:** PDFs were converted to images and processed using the `PIL` library. OCR was performed using `pytesseract` with a multi-attempt strategy (up to two attempts) to account for errors arising from quality variability.
- **LLM-based Extraction:** After OCR, the text was fed into a language model (LLM) via a custom prompt designed to extract product information and sender details. The prompt required:
  1. Providing a numbered list for each product.
  2. Outputting a line in the format `Sender: <company name>` (or `Sender: Not found` if unavailable).
  3. Outputting a line for additional company information in the format `SenderInfo: <additional details>`.

A simplified snippet of the code is provided below:

```python
def generate_prompt(ocr_text):
    instructions = (
        "You are an assistant that extracts product information from receipts. "
        "Follow these instructions: \n"
        "1) Provide a numbered list for each product. \n"
        "2) Output a line 'Sender: <company name>' (if not found, output 'Sender: Not found'). \n"
        "3) Output a line 'SenderInfo: <additional company details>' \n\n"
    )
    return f"{instructions}Receipt text:\n{ocr_text}"
```

Listing 4.1: Prompt Generation for LLM-based Extraction

4.2.3 Sampling Strategy and Benchmarking

4.2.3.1 Sampling Strategy

To evaluate the performance of different LLMs in extracting product information from the OCR-processed receipts, we randomly selected a statistically significant sample from the total dataset using Yamane's sample size formula (Yamane, 1967). Assuming a 95% confidence level (z = 1.96) and a conservative population proportion of 50% (p = 0.5), the minimum required sample size was calculated as:

$$n = \frac{N}{1 + N(e)^2}$$

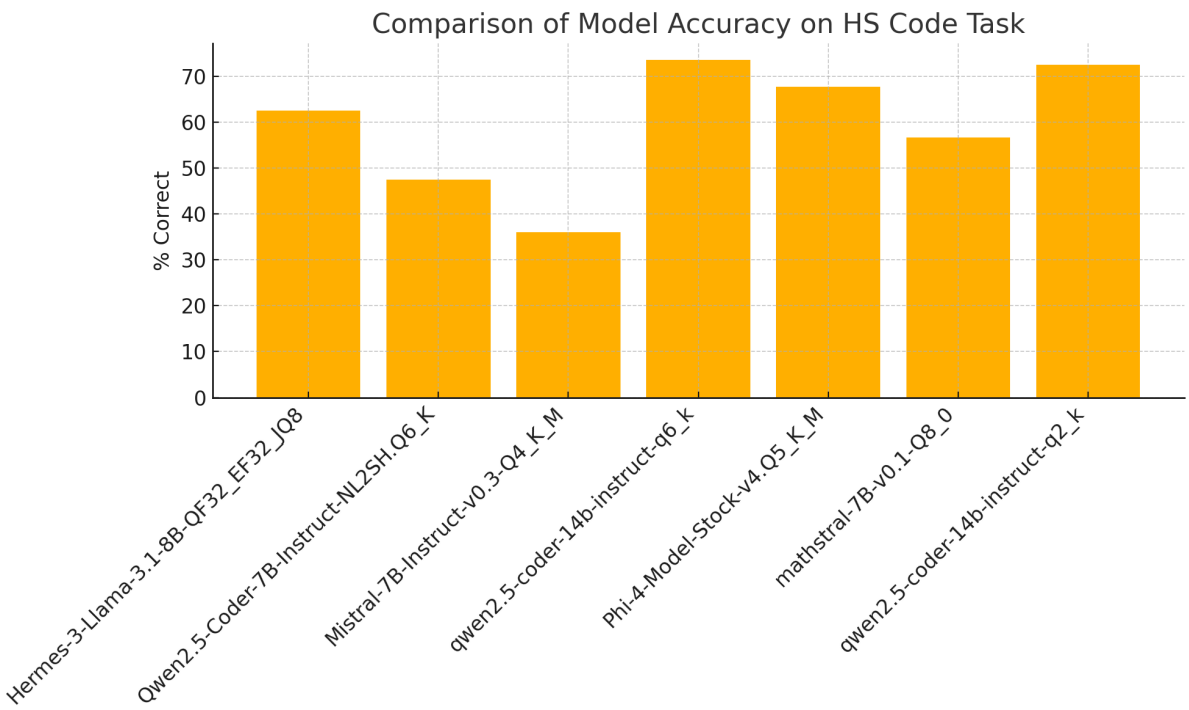
where n is the required sample size, N is the population size, and e is the desired margin of error (set to 5%).

With a combined dataset of 51,276 receipts (29,593 air freight and 21,683 sea freight), the resulting minimum sample size was 381. To ensure proportional representation, the sample was stratified based on shipment type:

- **Air Shipments:** Approximately 57.33% (218 receipts)
- **Sea Shipments:** Approximately 42.67% (163 receipts)

4.2.3.2 Benchmarking

Multiple OCR-LLM combinations were benchmarked on the sample by comparing computational time and error. Since even the sample contains a lot of products, a thorough manual comparison for every product was not feasible. The error was therefore found by comparing the amount of products the LLM suggested for every receipt against a manual count of product for every receipt. This gave us the following:



Based on these experiments, the model **qwen2.5-coder-14b-instruct-q6_k** was selected for having the lowest error as well as among the lowest computational time.

4.2.4 Structuring Data for Machine Learning Models

The final step involved organizing the cleaned and structured data into a standardized format for downstream machine learning tasks, including classification, clustering, and further extraction of product information. Application of the OCR pipeline to the full dataset yielded a CSV file with 212,919 product entries, containing the following fields:

- `TrackInfoId` (internal identification number at JetCarrier),
- `Sender` (sender name),
- `SenderInformation` (some senders have information about them - e.g. "largest automotive parts dealer"),
- `ProductId` (running counter for every product in one receipt),
- `ProductInformation` (the product description from the receipt),
- `PrelimHsCode` (JetCarrier's preliminary HS code),
- `FinalHsCode` (JetCarrier's final HS code - ground truth),
- `PredictedHsCode` (predicted codes from the tested systems),
- `ReceiptFile` (receipt identification - unique for every receipt)

4.2.5 Tools and Libraries

Category	**Libraries & Tools**
OCR & document processing	`pdf2image`, `Pillow`, `pytesseract`
HTTP / LLM communication	`requests`
Data handling & utilities	`os`, `time`, `json`, `csv`
External binaries	`poppler-utils` (e.g. `pdftoppm`), Tesseract CLI
External services	LM Studio (local REST API), `qwen2.5-coder-14b-instruct-q6_k` (GGUF model)

Table 4.1: Software stack used in the OCR pipeline

4.2.6 OCR Pipeline Overview

Figure 4.1 illustrates the complete pipeline implemented for processing receipt data through OCR and LLM-based information extraction. This pipeline was applied in two contexts: first, on a statistically sampled subset of 381 receipts to compare the performance of different LLMs; and second, on the entire dataset of 51,276 receipts using the best-performing LLM. The process begins by retrieving and preprocessing the receipts along with their corresponding metadata. The receipt images are cleaned to optimize OCR performance and then processed using Tesseract to extract text. This text is further cleaned and passed to an LLM to extract product and sender details. In the final stage, the extracted information from the receipts is combined with metadata to produce a structured CSV file.

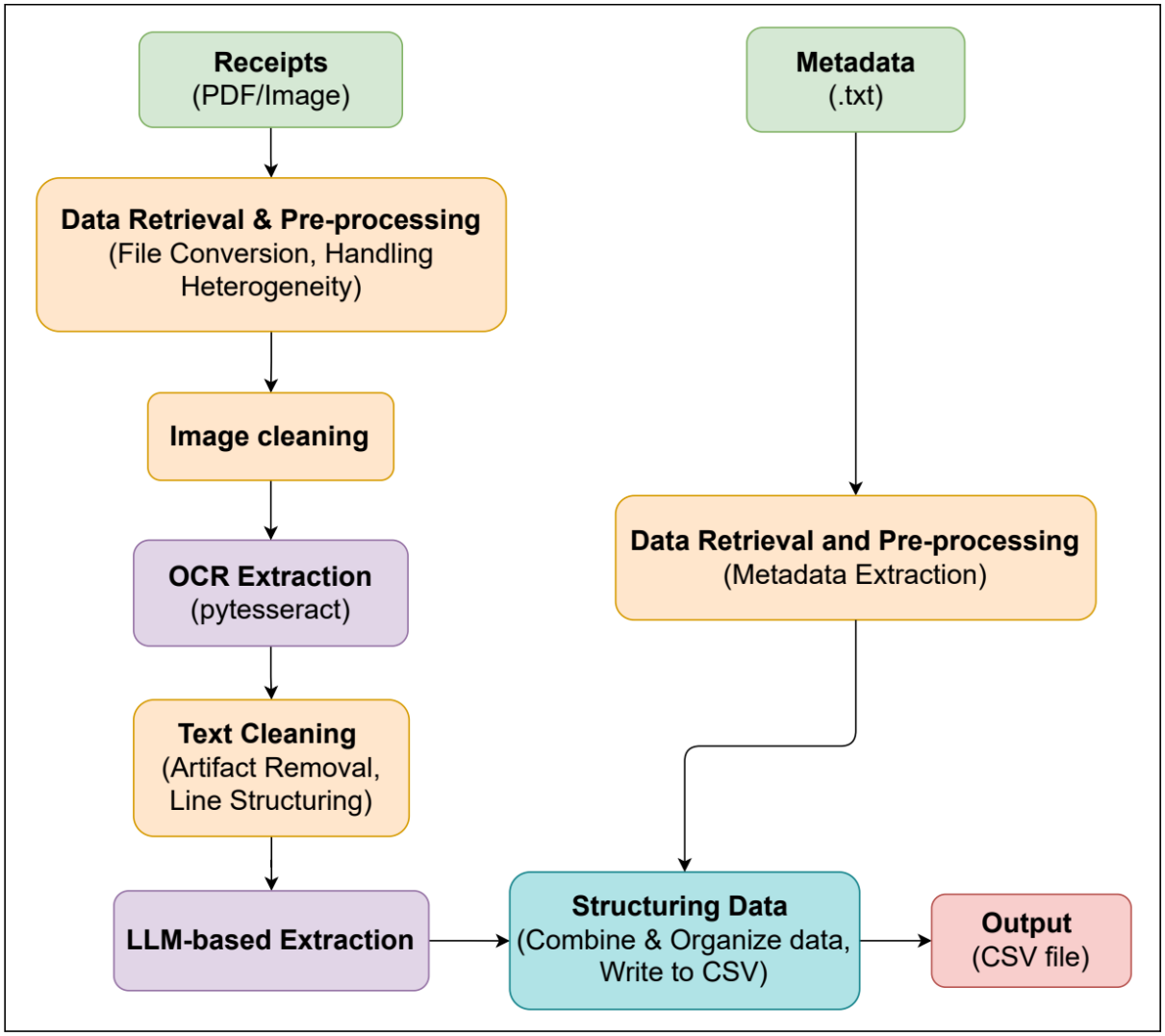


Figure 4.1: End-to-end pipeline for OCR and LLM-based information extraction from receipts

4.3 Part 2: HS Code Classification Implementation

4.3.1 Classic Supervised Classification

4.3.1.1 Preprocessing

First, the product descriptions were cleaned of noise. This means removing symbols, unneeded text, punctuation, and stop words. Then, apply spell correction, stemming, and lemmatization.

The SentenceTransformer framework was used with all-MiniLM-L7v2 on the cleaned text descriptions. It maps the product descriptions to a 384-dimensional vector space. This sentence-transformer was used as it is fast and intended for short sentences, like the product descriptions.

As the number of attributes increases, the need for more examples per output class grows accordingly. The training time for the models also grow with the number of attributes. To address this, Principal Component Analysis (PCA) was applied to reduce the vector dimensionality to 100.

4.3.1.2 Model Architecture

The models were structured throughout the hierarchy with a LCPN approach, with a classifier per parent node. This means that each classifier only takes into account the classes of the children for that parent node.

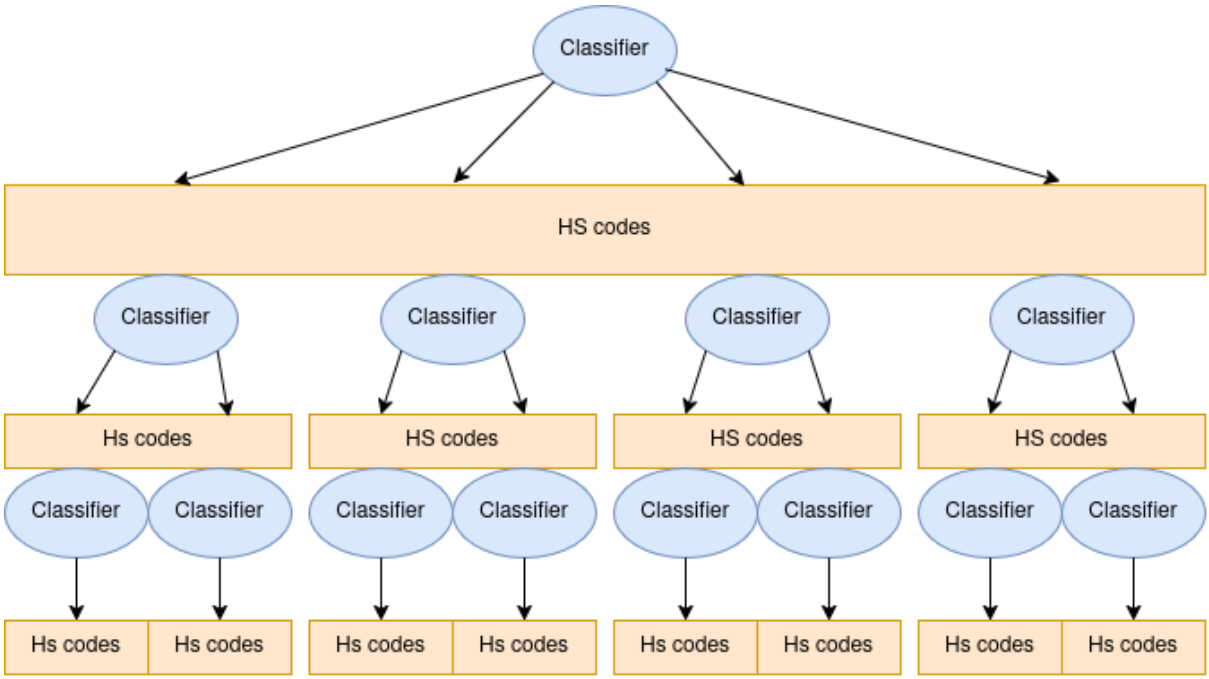


Figure 4.2: Supervised Classification Architecture

Several model types were trained on the dataset. There were two models from scikit-learn's framework, and two neural network models made with PyTorch. The models from scikit-learn's framework were RF and SVC. The PyTorch models were a simple FFNN and a small CNN.

A final test set of 400 examples was drawn from the dataset. This set was meant to be used during the testing phase. These examples were removed from the rest of the dataset that was used in the training phase.

When splitting data during the training phase, it was split into training, validation, and testing. The split was applied at every node. As in, the data that had the same parent node in the HS hierarchy.

The scikit models used 20% of the data for testing, and 80% for training. The neural networks used cross-validation, with 60% for training, 20% for validation, and 20% for testing.

4.3.1.3 Sample Distribution

A major problem with training the models was the amount of data in comparison to the number of classes. Each data example consisted of 100 features, so a lot of data for each class was needed to train a model.

The dataset from Jetcarrier consisted of 50.000 receipts. After OCR and removal of empty/non-sensical descriptions, the dataset consisted of over 200.000 product descriptions. The complete dataset also contains 4.357 different codes/classes in total. Many of the codes also have similar semantic content and may differ only slightly, e.g. based on materials or weight. This provides a substantial challenge, as the models/systems need have to both a high enough "input resolution" as well as contextual "understanding" to differentiate between such small differences. The

datasets used in similar research had millions of examples. Ruder (2020)'s dataset consisted of one million examples, with 3.000 unique HS codes. Altaheri (2019)'s dataset consisted of 22 million descriptions. Paramartha et al. (2021)'s dataset consists of 1.3 million records.

Many codes were not prevalent enough to train a robust model. To train a model for a node, a small threshold of 10 examples was needed.

100 models were trained throughout the hierarchy. There were 900 nodes where there were not enough data to meet the threshold.

The first model at the highest level in the HS code hierarchy was a single model that trained on 200.000 descriptions. But afterwards, the amount of samples rapidly diminishes. This table shows the sample distribution across the models at different hierarchy levels.

In the next level in the hierarchy, there were on average 1000 samples with a lot of variation. A few models had a much larger sample size than most of the models. For 8 digits there were on average 300 samples.

4.3.2 Semantic Similarity Comparison

The implementation of semantic similarity comparison investigates whether product descriptions can be matched to HS codes based purely on semantic closeness, without the need for task-specific training. We use a pretrained language model to embed both product descriptions and HS code definitions, and compute similarity scores to rank potential matches. This is a Zero-shot approach that does not require domain-specific training data.

4.3.2.1 Preprocessing

The product descriptions were preprocessed with a custom Python function that applies regular-expression (regex) filters to remove irrelevant tokens such as prices, currency symbols, transaction IDs, order numbers, and generic stopwords. This noise-reduction step is essential, as semantic-similarity methods are susceptible to irrelevant tokens in short, unstructured text inputs (Stassin et al., 2023). An example of the product descriptions before and after preprocessing can be seen in Figure 4.3 below.

The hierarchical JSON structure from Tolletaten Norwegian Customs (2025) was flattened to produce one descriptive sentence per HS code for the HS code definitions to facilitate a one-to-one comparison of embeddings.

4.3.2.2 Embedding

We used the SentenceTransformers framework to create embeddings for both product descriptions and the flattened HS code definitions from the lookup table. We chose the all-mpnet-base-v2 model from (Face, 2024) based on its strong performance in sentence-level semantic tasks as demonstrated by Stassin et al. (2023) in their paper.

4.3.2.3 Distance Calculations

We computed the cosine similarity between each product embedding and all HS code embeddings using `scipy`'s `cdist` function. Cosine distance was selected over alternatives like Euclidean or Manhattan distance based on findings from Stassin et al. (2023), who reported that cosine similarity provided the best top-k accuracy in their comparative evaluation of distance metrics for HS classification.

	ProductInformation	CleanedProductInformation
0	KITDTRD2 - Drop Trap $428,60 USD	kitdtrd drop trap
1	Product: Melton International Tackle, $47.99 U...	product melton international tackle kr nok
2	ARIES | 4042 Grille Guard / Bull Bar | SEK 332...	aries grille guard bull bar sek sek sek
3	Teal Maple Burl - 1 piece - USD 139.00	teal maple burl piece
4	**Product: Vintage 50s nos Coca Cola Visor lic...	product vintage s nos coca cola visor license ...
5	Original 1963 Cadillac Owner's Operator Instru...	original cadillac owner s operator instruction...
6	Item: MSD 8598 Ford 289/302 Billet Distributor...	item msd ford billet distributor ignition kit
7	Product: SMS Auto Fabrics	product sms auto fabrics
8	Super saeoe et Transmission | $2,069.95	super saeoe et transmission
9	Kohler K-26318-BL - Shower Components Bath and...	kohler k bl shower components bath and shower ...

Figure 4.3: Product descriptions before (left) and after preprocessing (right).

4.3.3 LLM Based System

4.3.3.1 System Overview

The LLM system works in three steps.

1. **Input processing:** We clean the product text and add short web snippets for enriching the product description.
2. **Candidate retrieval:** A TF–IDF model finds the K most similar HS descriptions.
3. **Code ranking:** An LLM looks at the full context and gives probabilities for the top N codes.

The LLM then makes the final, more exact choice.

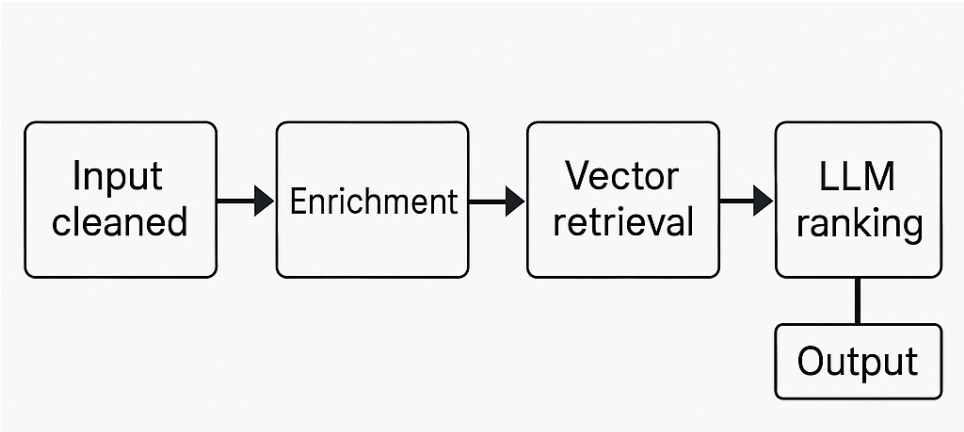


Figure 4.4: An overview of the LLM system

4.3.3.2 Input Cleaning and Normalisation

Rule-based filters

Receipts often hold prices, part numbers, and units that do not help the task. Regular expressions remove these items (Listing 4.2), trim spaces, and set all words to lower-case. We use `spaCy` to lemmatise with `en_core_web_sm`. If `spaCy` is not available, we just keep the lower-case words.

```
PRICE_RE = r"$?\d+(?:[.,]\d+)?\s*(?:USD|EUR|NOK|kr)?"
PART_RE  = r"\b[A-Z0-9]{5,}\b"      # Part numbers
UNIT_RE  = r"\b(?:kg|lbs?|cm|mm|pcs?)\b"
```

Listing 4.2: Key cleaning patterns.

Sender normalisation

We also strip common company endings such as "Ltd.", "Inc.", or "GmbH" from the sender name.

4.3.3.3 Contextual enrichment

Very "uninformative" product text, like "Exedy 6R80 EFK291HP2STL", can confuse the model. We therefore add:

- a 300-character DuckDuckGo snippet for *sender + product*, and
- a 300-character Wikipedia snippet for the product term.

Both calls time out after 10 s, so latency stays under control.

4.3.3.4 Vector-Space Retrieval

Corpus preparation

We flatten the Norwegian tariff tree to its 8-digit codes, but conserve the product description fom each previous "level" such that the complete context for the HS code is available for the model. After lemmatisation, a `TfidfVectorizer` is trained and saved as `tfidf_hs_vectorizer.pkl`.

Similarity search

At runtime we compare the query vector to the stored TF–IDF matrix with cosine similarity. We return the top $K = 50$ matches as tuples $(\text{code}, \text{description}, \text{similarity})$.

4.3.3.5 LLM-Based Probabilistic Ranking

Prompt design

The LLM sees:

1. A system message that tells it to "think" first.
2. A user prompt with:
    - the cleaned context,

- the 50 candidates with scores,
- a JSON schema for the answer.

This schema keeps the output machine-readable. newline

Model endpoint

We use `qwen3-14b` behind an OpenAI-style HTTP API on `localhost:1234`. We set `temperature` to 0.2 to make the model more predictable set and `max_tokens` to 20 000 to enable a large enough context window for the prompt and answer.

JSON parsing

If the model adds extra text, a regex finds the first JSON block and we parse it. After three failures we skip the row and log an error.

4.3.3.6 Batch Inference Pipeline

Algorithm 1 shows the full loop. On 300 receipts an RX 7900 GRE GPU and a Ryzen 7900X CPU need about 10 hours. This equates to around 2 minutes per product on average.

Algorithm 1 End-to-end HS code suggestion per product line

$q \leftarrow \textsc{CleanProduct}\big(r.\texttt{ProductInformation}\big)$
if $q = \emptyset$ **then return** $\textsc{None}$
end if
$s \leftarrow \textsc{CleanSender}\big(r.\texttt{Sender}\big)$
$e \leftarrow \mathrm{DDG}(s \| q) \parallel \textsc{Wiki}(q)$
$K \leftarrow \textsc{RetrieveCandidates}\big(q \| e,\, 50\big)$
$\mathcal{P} \leftarrow \textsc{RankCodes}(K, 5)$
append $\mathcal{P}$ to results

We then write the predictions to a CSV file as a JSON string in `PredictedHsCode`.

4.3.3.7 Tools and Libraries

Table 4.2: Software stack used in the HS code matching pipeline

Category	Libraries & Tools
NLP & text preprocessing	`spacy` (optional lemmatisation)
Vectorisation & similarity search	`scikit-learn` (`TfidfVectorizer`, `cosine_similarity`), `joblib`
Web requests & scraping	`requests`, `BeautifulSoup` (`bs4`)
Data handling & utilities	`pandas`, `json`, `os`, `pathlib`, `re`, `warnings`, `typing`
External services	DuckDuckGo HTML search, Wikipedia API, LM Studio (local REST API, `Qwen 3 14B GGUF model`)

4.3.3.8 Summary

The system joins fast vector search with LLM reasoning to give clear HS code suggestions and probabilities. Because each part is separate, we can improve cleaning, retrieval, or ranking

without touching the others.

4.3.4 NLP Based System

4.3.4.1 System Overview

The NLP-based system implements a fine-tuned BERT classifier that takes OCR-extracted product descriptions as input and outputs a predicted HS code. The model is trained to perform multi-class classification over a set of HS code labels derived from Norwegian Customs data.

4.3.4.2 Tokenization and Input Encoding

The cleaner product descriptions are tokenised using the pre-trained "bert-base-uncased" tokenizer. Each input is converted into a sequence of token IDs and attention masks. These token sequences are then padded and truncated to a fixed length (128 tokens) to ensure consistent batch shapes. The tokenized data is wrapped into a PyTorch "Dataset" object and loaded with a dataloader for GPU-based training.

4.3.4.3 Model Architecture and Training

The core model is "BertForSequenceClassification", this model extends the BERT base architecture with an added classification head, which is a linear layer for mapping the CLS token representation to the output space of HS code classes.

We trained the model by using a cross-entropy loss function, optimized with AdamW. The logits from the classifier head are passed through a softmax function to obtain class probabilities. We applied dropout during training to mitigate overfitting.

4.3.4.4 Prediction and Evaluation Logic

At inference, each product description undergoes the same preprocessing and tokenization steps as were done during training. The model outputs a vector of logits, one per class, from which the top-k classes are selected. Accuracy is measured using the top-3 and top-5 metrics. Additionally, predicted and ground truth labels are truncated to 6-digit and 3-digit prefixes to evaluate how well the model identifies general product categories.

Chapter 5

Results and Discussion

5.1 Results

To test the different solutions for suggesting HS codes, we created a test dataset by using Yamane's formula again. That gave us a test dataset with 400 products. The products were selected from the complete OCR csv-file that was generated ($\approx$ 213000 products/rows). The sample was excluded from the training and validation dataset for the models. Since the data from JetCarrier only gave one HS code per receipt (ground truth), and not one HS code per product, we selected products from receipts with only one product to make sure that the provided HS code is as accurate as possible. This gave us 400 receipts where 98 of them did not include a product or had product descriptions that clearly did not include a product (e.g. "Parts - Quantity: 1, Price: 127.90 USD, Amount: 127.90 USD"). Since no description or clearly wrong descriptions would be uninteresting for the testing of the different solutions, we excluded these from the final test dataset, leaving us 302 receipts/products in the test dataset.

The test was conducted by analyzing the top 3 and 5 suggested HS codes on the first 3, 6 and 8 digit codes respectively. If one of the top 3 or 5 codes matches the ground truth as provided by JetCarrier, it was regarded as a "hit".

To better understand the results from the LLM-based system, we also tested the vector model for this system on the dataset. The vector model was also tested for top 50, since this was the maximum number of suggestions from the vector model for the LLM system.

5.1.1 Results for 3-digits

Table 5.1: 3-digit accuracy (%) by model and suggestion threshold

Model	Top 3	Top 5	Top 50
CNN	78	85.10	–
FFNN	84.50	89.80	–
LLM	24.17	27.15	–
NLP	81.94	85.28	–
Random Forest	61.80	70	–
SVM	62	71.10	–
SentenceTransformer (SSC)	35.10	39.74	–
Vector model for LLM	14.29	16.94	41.86

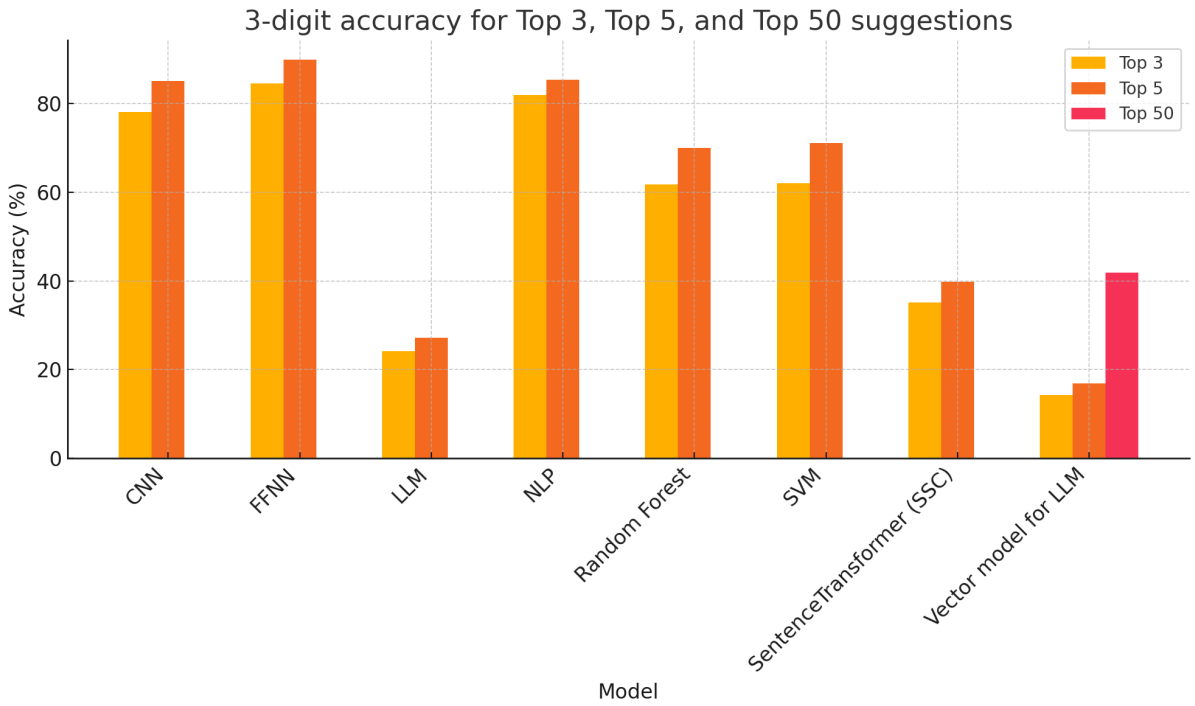


Figure 5.1: Accuracy for 3-digit level for all models under Top 3, Top 5 and (for the LLM vector model) Top 50.

3-digit accuracy (Figure 5.1). The domain-specific sequence models achieve the highest hit rates at this level of code granularity. A feedforward neural network (FFNN) reaches approximately 85 % accuracy for Top 3 and 90 % for Top 5, closely followed by the NLP-based approach at 82 %/86 % and the CNN at 78 %/85 %. Traditional machine-learning methods such as Random Forest and SVM perform on par with the CNN. General LLM outputs (without vector indexing) lag behind at 24 %/27 %, while the SentenceTransformer (SSC) model attains 62 %/71 %. The vector-based LLM method starts at a modest 14 %/17 % (Top 3/Top 5), but when allowed up to 50 suggestions its hit rate increases to 42 %.

5.1.2 Results for 6-digits

Table 5.2: 6-digit accuracy (%) by model and suggestion threshold

Model	Top 3	Top 5	Top 50
CNN	0.20	0.20	–
FFNN	0.20	0.20	–
LLM	13.25	14.57	–
NLP	71.91	76.92	–
Random Forest	0.20	0.20	–
SVM	0	0	–
SentenceTransformer (SSC)	11.59	16.56	–
Vector model for LLM	6.31	7.97	18.94

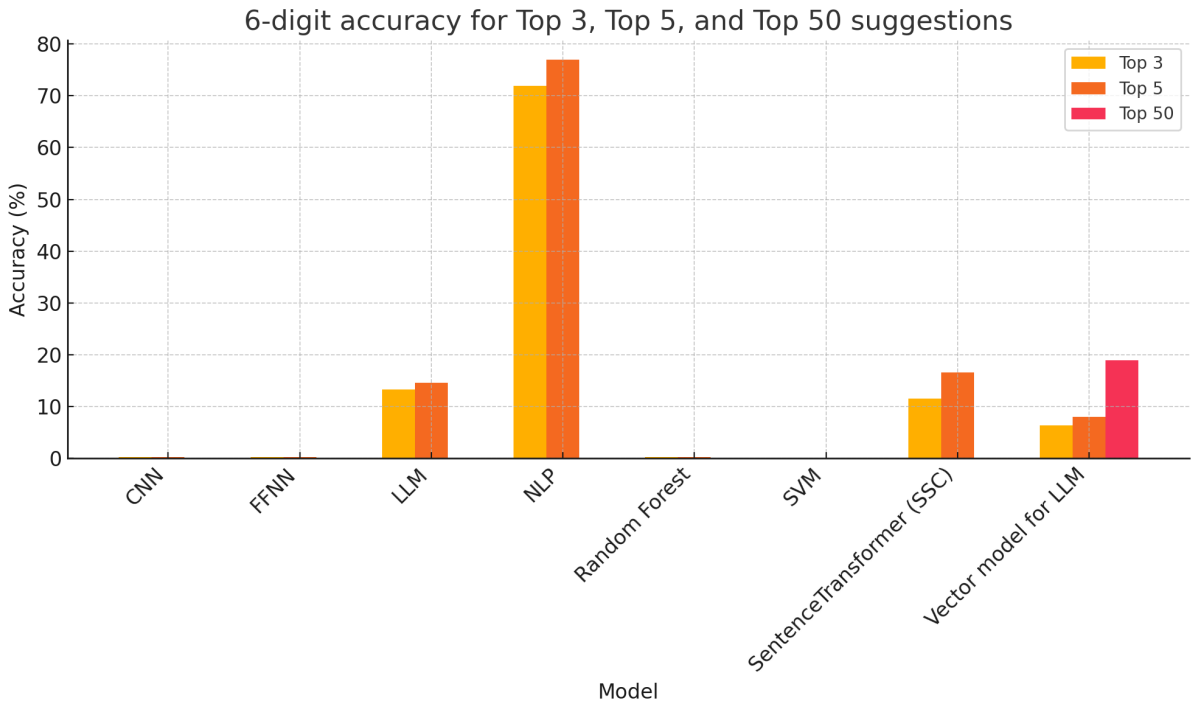


Figure 5.2: Accuracy for 6-digit level for all models under Top 3, Top 5 and (for the LLM vector model) Top 50.

6-digit accuracy (Figure 5.2). As code specificity increases, most image- and feature-based models collapse toward zero accuracy, with the exception of the NLP approach, which maintains 72 % for Top 3 and 77 % for Top 5. The FFNN, CNN, RF and SVM models cease to work at 6-digit, as there are too few examples for each label to train the models. The LLM model reaches only 13 %/15 %, and the SentenceTransformer 12 %/17 %. The vector LLM variant scores 6 %/8 % (Top 3/Top 5) and improves to 19 % when extended to Top 50 suggestions.

5.1.3 Results for 8-digits

Table 5.3: 8-digit accuracy (%) by model and suggestion threshold

Model	Top 3	Top 5	Top 50
CNN	0	0	–
FFNN	0	0	–
LLM	9.93	11.26	–
NLP	69.90	75.25	–
Random Forest	0	0	–
SVM	0	0	–
SentenceTransformer (SSC)	7.28	11.93	–
Vector model for LLM	4.65	6.31	16.61

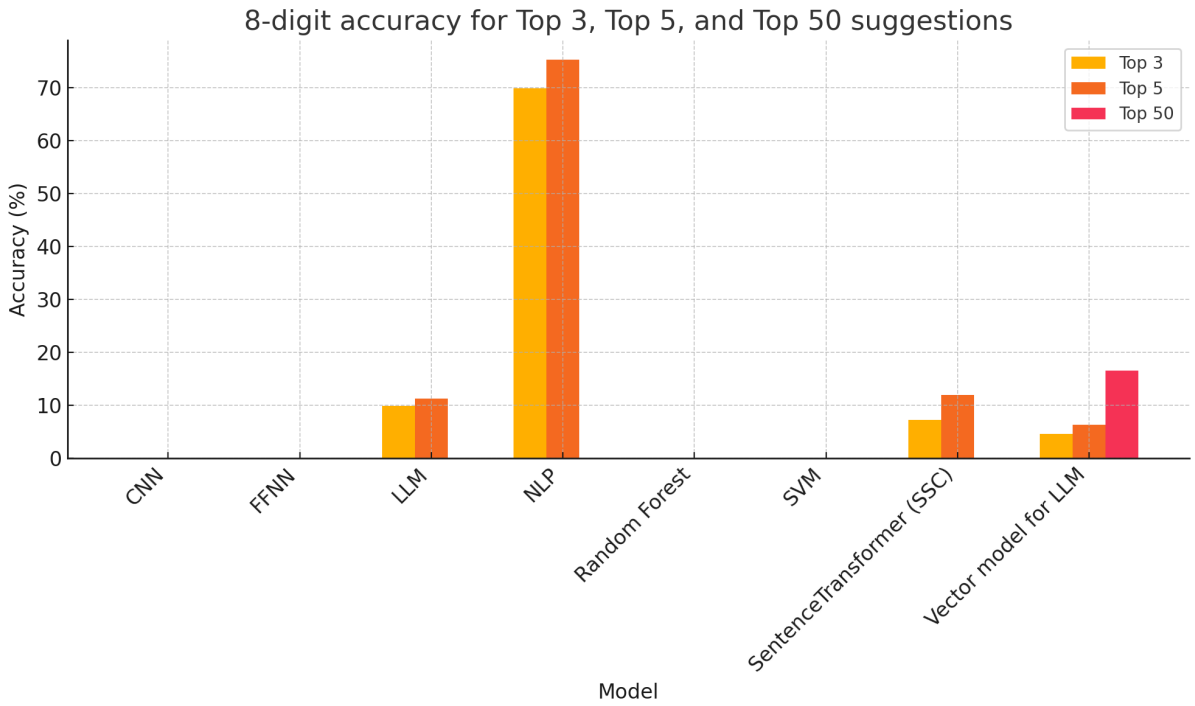


Figure 5.3: Accuracy for 8-digit level for all models under Top 3, Top 5 and (for the LLM vector model) Top 50.

8-digit accuracy (Figure 5.3). The pattern observed at six digits persists. The NLP model remains robust with 70 %/75 % (Top 3/Top 5), whereas CNN, FFNN, Random Forest and SVM have ceased to work. The general LLM solution reaches about 10 %/11 %, SentenceTransformer 7 %/12 %, and the vector LLM variant 5 %/6 % (Top 3/Top 5), rising to 17 % under a Top 50 scheme.

5.1.4 Summary

Overall, a domain-tuned NLP sequence model consistently outperforms all other approaches across code granularities. Allowing more suggestions (Top 5 vs. Top 3) yields modest gains (4–6 %) for most methods. The vector-based LLM benefits substantially from a larger suggestion window (Top 50), yet remains below the performance of the best sequence models. Supervised classification works well when there is enough data, such at 3-digit, but it needs a lot of data per label to work properly. Finally, as HS code specificity increases, semantic language understanding becomes increasingly critical compared to purely image- or feature-based classification.

5.2 Discussion

The results demonstrate clear advantages and trade-offs among the different HS-code suggestion approaches. First, the domain-tuned NLP sequence model consistently outperforms all other methods at every level of code granularity. This suggests that explicit feature extraction and task-specific training remain highly effective for structured classification tasks such as HS-code prediction, where the model can learn patterns in product descriptions and map them directly to code hierarchies. This is however a somewhat surprising result as the product descriptions often lack a meaningful semantic relationship with the HS code description. This does however suggest that there has to be an underlying relationship that is not easy for humans to spot.

In contrast, general-purpose LLM outputs (without vector indexing) and image- or feature-

based models (CNN, FFNN, Random Forest, SVM) struggle except at the coarsest (3-digit) level. The poor performance of these approaches on 6- and 8-digit codes indicates that they lack the fine-grained contextual understanding necessary for precise code assignment, and that the supervised classification approach did not have enough data to train models for these digits. Even the SentenceTransformer embeddings fall short of the domain-tuned NLP model when the task requires deeper semantic distinctions.

When comparing the LLM results with the vector model for the LLM, we can see that it performs ≈twice as good as the vector model for top 3 and 5. It does however perform worse than the top 50 for the vector model, meaning that the LLM fails to find the correct HS code in many cases even though it is presented with the correct code in the prompt. A better model may perform much better than the tested model.

Several limitations temper these conclusions. Our test set—302 single-product receipts derived via Yamane's sampling—may not fully capture the diversity of real invoices, especially multi-line items or ambiguous descriptions. Moreover, we assume perfect labeling from JetCarrier, which may not be the case in an organic dataset like this. In addition, we know that the OCR pipeline that we created has weaknesses, so even after cleaning our test dataset by removing obvious errors, the remaining products will contain instances of "weak" descriptions. This is a general challenge when using product descriptions from receipts that are not designed to capture the semantic relationship with the HS classification system.

Chapter 6

Conclusion and Future Work

6.1 Conclusion

This project showed that automating HS code predictions of the 4357 HS codes in Norway is viable, but performance relies on method, enough data, and data quality. Our fine-tuned BERT NLP model was the only method that achieved a high top-3 and top-5 accuracy across the complete 8-digit HS code, scoring roughly 82% on top-3 accuracy and 86% on top-5 accuracy. Classical supervised models matched or beat the NLP model at 3-digit predictions but collapsed to near-zero accuracy for 6-digit and 8-digit HS code predictions. The Semantic Similarity Comparison and LLM implementations worked without pre-training, showing promise for a low-maintenance automated HS code prediction system. However, the models struggle with the poor quality of the product descriptions from the receipt, which sometimes contain as little as a single word, leaving too little semantic context for these models to assign an accurate HS code. The lack of high-quality data functions as a bottleneck for improved prediction accuracy and is the most direct path to higher performance for JetCarrier. This also underscore the need for a better OCR system and, preferably, a better way for JetCarrier to retrieve meaningful product descriptions. A mere receipt based system for gathering product descriptions may not be viable for a system to work in practice. Future work should also focus on gathering information on precision and recall as well as accuracy.

6.2 Future Work

Future work should evaluate hybrid architectures that combine the precision of domain-tuned sequence models with the flexibility of LLM embeddings—perhaps by using NLP outputs to re-rank vector-retrieved candidates. Expanding the dataset to include multi-product receipts and incorporating user feedback loops would also improve real-world robustness. Further more, a better OCR solution should be looked into, and JetCarrier should investigate how they can retrieve product descriptions that are more tailored to the HS classification system.

Finally, a cost-benefit analysis of candidate list length versus manual correction effort could guide optimal threshold selection in production settings.

References

Altaheri, F. A. M. A. (2019). *q* (Master's thesis). The British University in Dubai, Dubai, UAE.

Anggoro, A. W., Corcoran, P., De Widt, D., & Li, Y. (2023). Using distilbert to assign hs codes to international trading transactions.

Association for Computing Machinery. (2024). *About the acm digital library.* Retrieved from `https://dl.acm.org/about` (Accessed May 12, 2025)

Awel, M. A., & Abidi, A. I. (2019). Review on optical character recognition. *International Research Journal of Engineering and Technology (IRJET)*, *6*(6), 3666–3669. Retrieved from `https://www.irjet.net/archives/V6/i6/IRJET-V6I6736.pdf`

Costa, L., Oliveira, I., & Fileto, R. (2023, 03). Text classification using embeddings: a survey. *Knowledge and Information Systems*, *65*, 1-43. doi: 10.1007/s10115-023-01856-z

de Lima, R. R., Fernandes, A. M. R., Bombasar, J. R., da Silva, B. A., Crocker, P., & Leithardt, V. R. Q. (2022). An empirical comparison of portuguese and multilingual bert models for auto-classification of ncm codes in international trade. *Big Data and Cognitive Computing*, *6*(1). Retrieved from `https://www.mdpi.com/2504-2289/6/1/8` doi: 10.3390/bdcc6010008

Elsevier. (2024). *Sciencedirect: Elsevier's premier platform of peer-reviewed scholarly literature.* Retrieved from `https://www.elsevier.com/products/sciencedirect` (Accessed May 12, 2025)

Face, H. (2024). *Mpnet — hugging face transformers documentation.* Retrieved from `https://huggingface.co/docs/transformers/en/model_doc/mpnet` (Accessed: 2025-05-13)

Google. (2024). *Google scholar.* Retrieved from `https://scholar.google.com/` (Accessed May 12, 2025)

Ha, H., & Horák, A. (2022). Information extraction from scanned invoice images using text analysis and layout features. *Signal Processing: Image Communication*, *102*, 116601. Retrieved from `https://www.sciencedirect.com/science/article/pii/S0923596521003015` doi: https://doi.org/10.1016/j.image.2021.116601

Institute of Electrical and Electronics Engineers. (2024). *About ieee xplore.* Retrieved from `https://ieeexplore.ieee.org/Xplorehelp/overview-of-ieee-xplore/about-ieee-xplore` (Accessed May 12, 2025)

Islam, N., Islam, Z., & Noor, N. (2017). *A survey on optical character recognition system.* Retrieved from `https://arxiv.org/abs/1710.05703`

JetCarrier. (2025). *Om oss.* Retrieved from `https://www.jetcarrier.com/cms/support/om-oss/` (Retrieved January 27, 2025)

Loukil, F., Cadereau, S., Verjus, H., Galfre, M., Salamatian, K., Telisson, D., ... Le Van, O. (2024). Llm-centric pipeline for information extraction from invoices. In *2024 2nd international conference on foundation and large language models (fllm)* (p. 569-575). Retrieved from https://ieeexplore.ieee.org/abstract/document/10852504 (Available at: https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=10852504) doi: 10.1109/FLLM63129.2024.10852504

Marra de Artiñano, I., Riottini Depetris, F., & Volpe Martincus, C. (2023). Automatic product classification in international trade: Machine learning and large language models. Retrieved from https://doi.org/10.18235/0005012 doi: https://doi.org/10.18235/0005012

Navasardyan, Z. (2024, May). Interpretable and generalizable hts code classification framework. *Economics, Finance and Accounting*, *1*(13), 140. Retrieved from https://efa.am/index.php/efa/article/view/62 doi: 10.59503/29538009-2024.1.13-140

Oliveira, V. D., Bezerra, Y. F., Weigang, L., Brom, P. C., & Celestino, V. R. R. (2024). *Slim-raft: A novel fine-tuning approach to improve cross-linguistic performance for mercosur common nomenclature.* Retrieved from https://arxiv.org/abs/2408.03936

Paramartha, I. G., Amaludin, B., & Wahana, A. (2021, 03). Hs code prediction tool using machine learning.

Raiaan, M. A. K., Mukta, M. S. H., Fatema, K., Fahad, N. M., Sakib, S., Mim, M. M. J., ... Azam, S. (2024). A review on large language models: Architectures, applications, taxonomies, open issues and challenges. *IEEE Access*, *12*, 26839-26874. Retrieved from https://ieeexplore.ieee.org/document/10433480 doi: 10.1109/ACCESS.2024.3365742

Ruder, D. (2020). *Application of machine learning for automated hs-6 code assignment* (MSc thesis). Tallinn University of Technology, Tallinn, Estonia.

Salvatore Rollino Subiabre, N. (2024). *Automating the extraction of data from printed receipts using machine learning and computer vision* (Master's thesis, KTH, School of Electrical Engineering and Computer Science (EECS)). Retrieved from https://www.diva-portal.org/smash/record.jsf?pid=diva2:1939993 (Available at: https://www.diva-portal.org/smash/get/diva2:1939993/FULLTEXT01.pdf)

Singh, A., & Diao, Y. (2024, December). Leveraging large language models for optimized item categorization using unspsc taxonomy. *International Journal on Cybernetics amp; Informatics*, *13*(6), 01–10. Retrieved from http://dx.doi.org/10.5121/ijci.2024.130601 doi: 10.5121/ijci.2024.130601

Spichakova, M., & Haav, H.-M. (2020). Application of machine learning for assessment of hs code correctness. *Baltic Journal of Modern Computing*, *8*(4), 698–718. doi: 10.22364/bjmc.2020.8.4.13

Springer. (2024). *Journals.* Retrieved from https://www.springeropen.com/journals (Accessed May 12, 2025)

Stassin, S., Amel, O., Mahmoudi, S. A., & Siebert, X. (2023). Similarity versus supervision: Best approaches for hs code prediction. In *Proceedings of the european symposium on artificial neural networks, computational intelligence and machine learning (esann 2023)* (pp. 175–180). Bruges, Belgium and online: i6doc.com. Retrieved from http://www.i6doc.com/en/

Tolletaten Norwegian Customs. (2025). *Customs tariff structure.* Retrieved from https://data.toll.no/dataset/customstariffstructure (Published January 8, 2025. Retrieved March 9, 2025)

Yamane, T. (1967). *Statistics: An introductory analysis* (2nd ed.). New York: Harper and Row.

Yiqiu Shen, J. E. K. D. H. B. R. G. S. L. M., Laura Heacock. (2023). *Chatgpt and other large language models are double-edged swords.*

Zhao, W. X., Zhou, K., Li, J., Tang, T., Wang, X., Hou, Y., ... Wen, J.-R. (2025). *A survey of large language models.* Retrieved from `https://arxiv.org/abs/2303.18223`